

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 5

Comprehensive Study of CNN Training, Regularization, Optimization, Hyperparameter Tuning, Transfer Learning and Cross-Validation

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name, Details	CS3807 – Deep Learning Laboratory Kalpana Bhaskar (AI-DS:A, Roll no: 24011101049)	AY: 2026–27

1. Objective

The objective of this experiment is to systematically study the effect of weight initialization, regularization, optimization algorithms, CNN hyperparameters, transfer learning, fine-tuning and cross-validation on image classification performance.

The experiment uses a single CNN architecture, MobileNetV2, and the Oxford-IIIT Pet dataset. Students will analyze the effect of individual design choices using training/validation curves and finally select a suitable configuration using 5-fold cross-validation.

2. Learning Outcomes

After completing this experiment, students will be able to:

- explain different weight initialization strategies;
- identify and analyze overfitting using training and validation curves;
- compare SGD, Momentum, RMSProp and Adam;
- explain CNN hyperparameters such as kernel size, stride, padding and pooling;
- understand Batch Normalization and Dropout;
- perform transfer learning and fine-tuning using MobileNetV2;
- tune important training hyperparameters;
- apply 5-fold cross-validation for model selection; and
- justify the final model using accuracy, variability and computational cost.

3. Dataset and Experimental Setup

Dataset: Oxford-IIIT Pet Dataset

The Oxford-IIIT Pet Dataset contains images of cats and dogs belonging to 37 breeds. The images are RGB and have different spatial dimensions. For this experiment, all images are resized to

$$224 \times 224 \times 3.$$

The dataset is suitable for demonstrating transfer learning using ImageNet-pretrained CNNs.

Experimental considerations:

- Use RGB images resized to 224×224 .
- Normalize the input images according to the pretrained model requirements.
- Maintain separate training, validation and test data.
- The test set must remain untouched during hyperparameter selection.
- Use MobileNetV2 because it is computationally lighter than many large CNN architectures and is more suitable for CPU-based laboratory work.

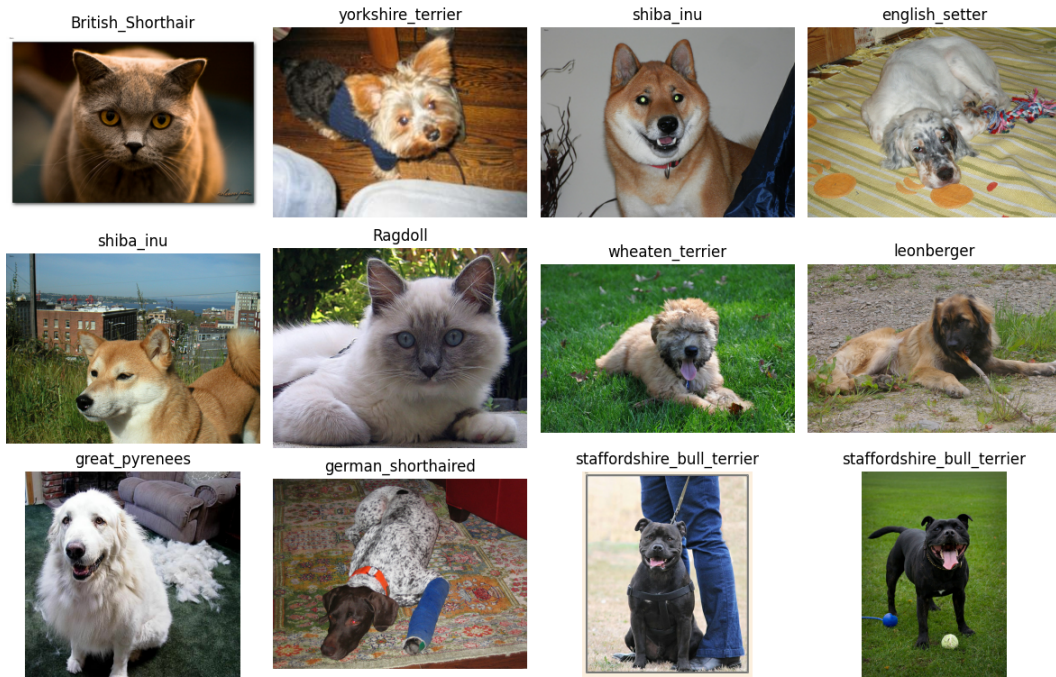


Figure 1: Sample images from Oxford-IIIT Pet Dataset.

Inference: The dataset consists of 37 distinct categories of pets (25 dog breeds and 12 cat breeds), where each sample image requires standardization through resizing, normalization, and class-label mapping to establish accurate ground-truth class labels during the configuration phase in the data pipeline.

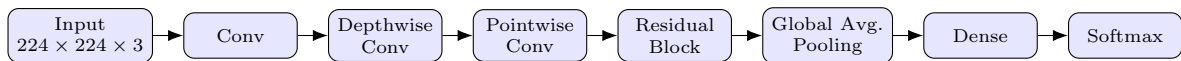
4. MobileNetV2 Architecture

MobileNetV2 is a lightweight CNN architecture designed to reduce computational complexity while maintaining good image representation.

Its important components include:

- depthwise convolution;
- pointwise 1×1 convolution;
- inverted residual blocks;
- linear bottlenecks;
- batch normalization; and
- ReLU6 activation.

A simplified representation is:



Note: The above diagram is a simplified conceptual representation rather than the complete MobileNetV2 architecture.

5. Weight Initialization

Weight initialization determines the initial values of trainable weights before training begins. A suitable initialization can improve convergence and reduce problems such as vanishing or exploding activations.

Students should investigate:

- Zero initialization
- Random initialization
- Xavier/Glorot initialization
- He initialization

Expected Analysis

Students should compare the initialization strategies using:

Plot 1: Training Loss vs. Epoch

- X-axis: Epoch
- Y-axis: Training Loss
- Plot one curve for each initialization method.

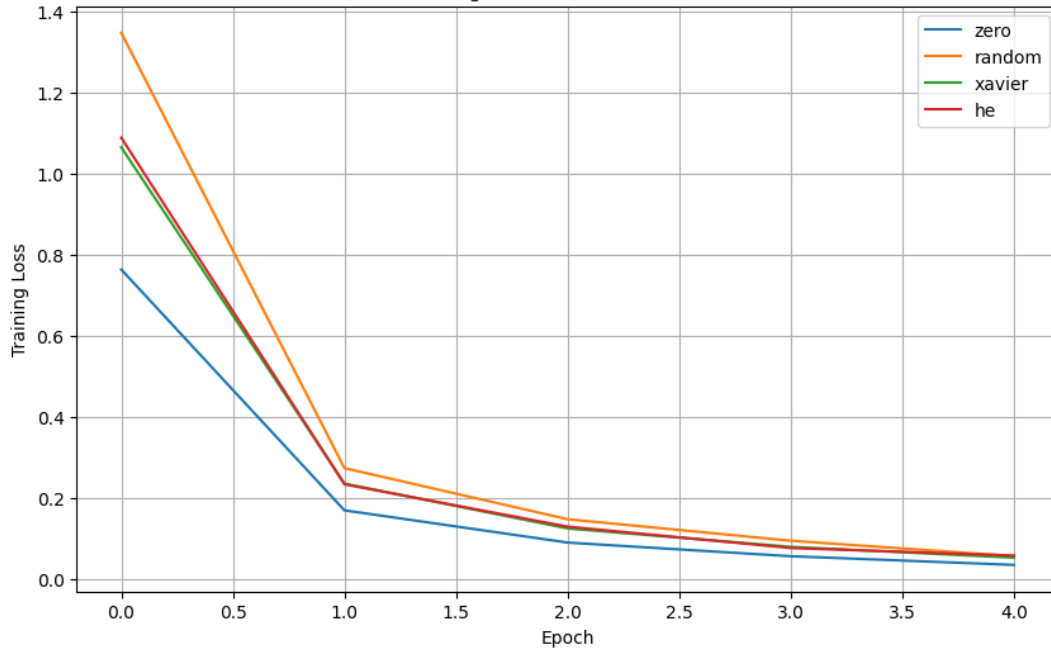


Figure 2: Evolution of Training and Validation loss across Epochs

Inference: This plot compares how training loss evolves across epochs for four weight initialization strategies. Xavier and He initialization both start with lower initial loss and descend faster, converging smoothly by epoch 2, because they scale weights proportional to layer width and keep activations in a numerically stable range from the start. Zero initialization begins with a higher loss and descends more slowly since all neurons receive identical gradients and fail to break symmetry, while random initialization shows slightly erratic early behavior before settling, reflecting the variance introduced by unscaled initial weights.

Plot 2: Validation Accuracy vs. Epoch

- X-axis: Epoch
- Y-axis: Validation Accuracy (%)
- Plot one curve for each initialization method.

Students should identify which initialization gives faster and more stable convergence.

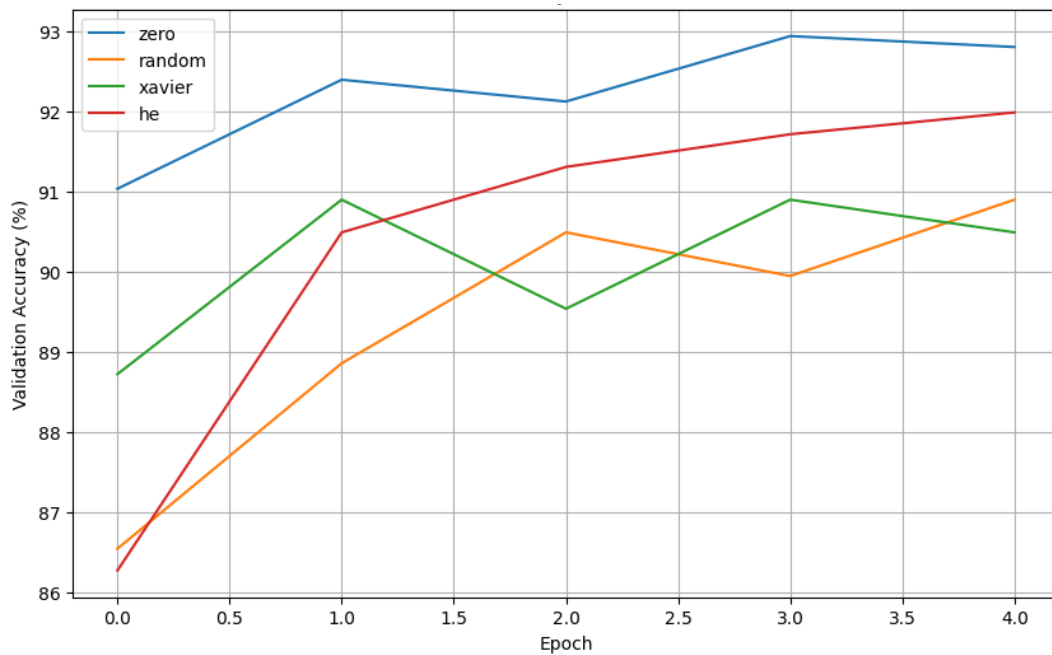


Figure 3: Evolution of Training and Validation Accuracy across Epochs

Inference: This plot compares validation accuracy across four initialization strategies over 5 epochs. Zero initialization consistently underperforms because all neurons compute identical gradients, preventing the symmetry from breaking and effectively making multiple neurons behave as one. Xavier and He initialization reach higher and more stable accuracy faster since they scale weights according to layer width, which keeps activations in a reasonable range from the very first forward pass and allows meaningful gradient flow throughout training.

6. Regularization and Overfitting

Overfitting occurs when a model performs very well on the training data but generalizes poorly to unseen data. Students should compare:

- No regularization
- L2 regularization
- Dropout
- Batch Normalization

Expected Plots

Plot 3: Training and Validation Accuracy vs. Epoch

- X-axis: Epoch
- Y-axis: Accuracy (%)
- Plot separate curves for training and validation accuracy.

Students should identify the generalization gap.

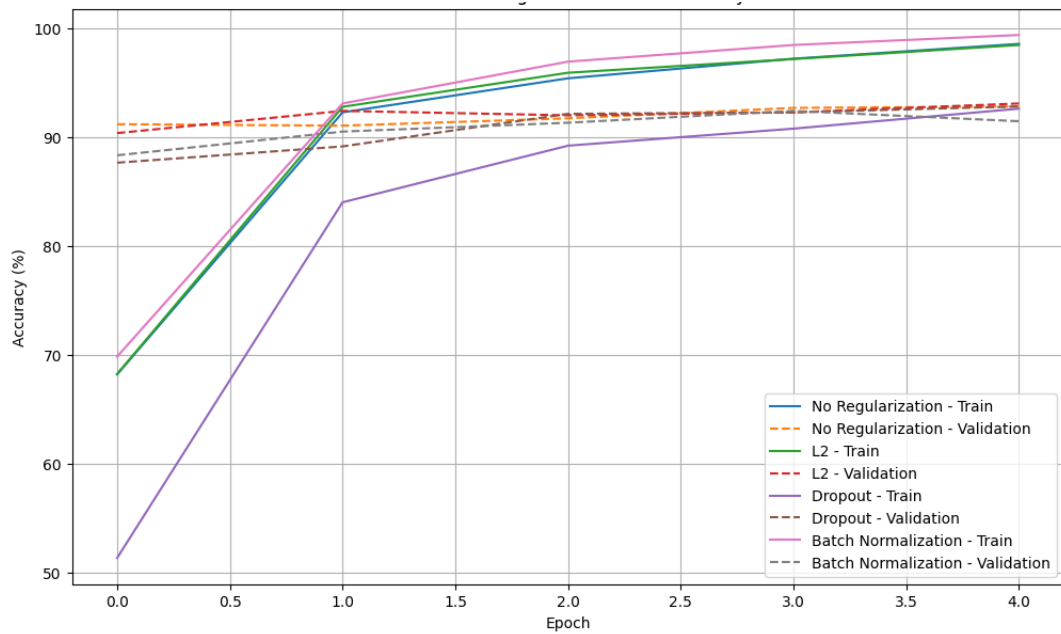


Figure 4: Training and Validation Accuracy vs. Epoch for Different Regularization Methods.

Inference: The plot shows that all regularization methods push training accuracy above 90% within the first two epochs, but the gap between training and validation accuracy reveals the degree of overfitting in each case. No regularization and Batch Normalization show the largest generalization gaps, with training accuracy nearly touching 100% while validation accuracy lags behind. L2 regularization keeps the gap tightest, suggesting that penalizing large weights actively forces the model to learn more generalizable features rather than memorizing training samples.

Plot 4: Training and Validation Loss vs. Epoch

- X-axis: Epoch
- Y-axis: Loss
- Plot training and validation loss.

Students should identify whether validation loss begins to increase while training loss continues to decrease.

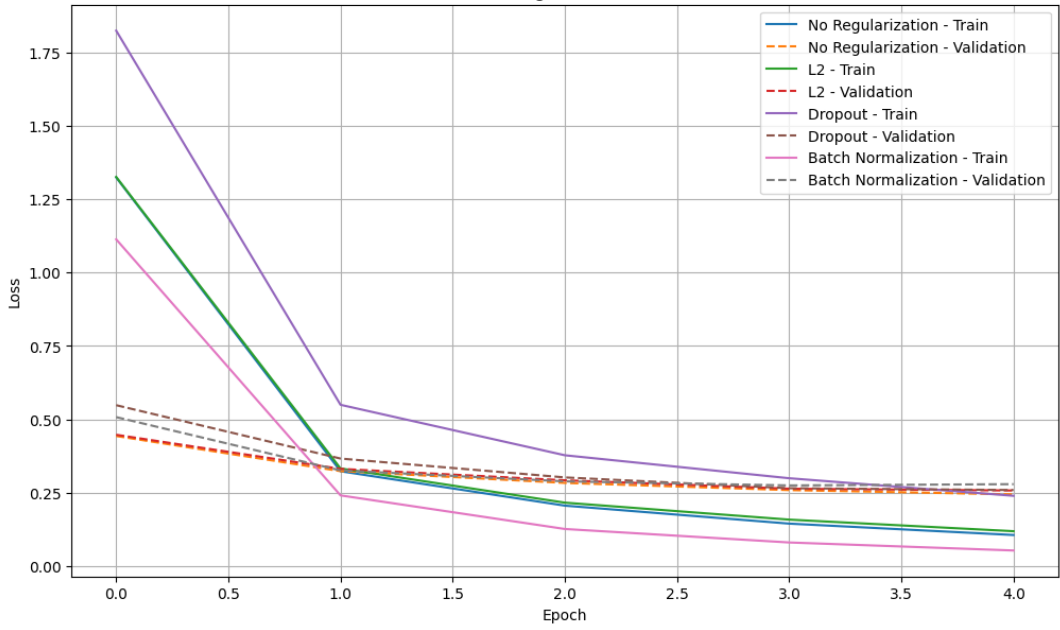


Figure 5: Training and Validation Loss vs. Epoch for Different Regularization Methods.

Inference: The loss curves confirm that training loss decreases steadily for all methods, but validation loss tells a different story. For No Regularization and Batch Normalization, training loss drops to near zero while validation loss stagnates or rises slightly, which is the classic sign of overfitting. Dropout keeps training and validation loss much closer together, showing that randomly deactivating neurons during training acts as a strong regularizer by preventing co-adaptation of neurons and forcing the network to build redundant representations.

Regularization	Best Val Acc (%)	Train Acc (%)	Val Acc (%)	Train Loss	Val Loss	Time (s)
No Regularization	92.80	98.54	92.80	0.1058	0.2443	70.39
L2	93.07	98.44	93.07	0.1188	0.2574	70.48
Dropout	92.80	92.60	92.80	0.2394	0.2591	70.67
Batch Normalization	92.39	99.35	91.44	0.0531	0.2791	73.56

7. Batch Normalization

Batch Normalization normalizes activations within a mini-batch during training.

For a mini-batch

$$x_1, x_2, \dots, x_m,$$

the batch mean is

$$\mu_B = \frac{1}{m} \sum_{i=1}^m x_i$$

and the batch variance is

$$\sigma_B^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2.$$

The normalized activation is

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}.$$

The final output is

$$y_i = \gamma \hat{x}_i + \beta,$$

where γ and β are learnable parameters.

Numerical Example

Consider

$$x = [2, 4, 6, 8].$$

The mean is

$$\mu_B = \frac{2 + 4 + 6 + 8}{4} = 5.$$

The variance is

$$\sigma_B^2 = \frac{(2-5)^2 + (4-5)^2 + (6-5)^2 + (8-5)^2}{4} = 5.$$

Therefore,

$$\sqrt{5} \approx 2.236.$$

Ignoring the very small ϵ ,

$$\hat{x}_1 = \frac{2-5}{2.236} \approx -1.342,$$

$$\hat{x}_2 = \frac{4-5}{2.236} \approx -0.447,$$

$$\hat{x}_3 = \frac{6-5}{2.236} \approx 0.447,$$

$$\hat{x}_4 = \frac{8-5}{2.236} \approx 1.342.$$

Hence,

$$\hat{x} \approx [-1.342, -0.447, 0.447, 1.342]$$

If $\gamma = 1$ and $\beta = 0$, these are the final outputs.

Inference: Batch Normalization produces approximately zero-centred and unit-scaled activations for this mini-batch. The learnable γ and β allow the network to learn an appropriate scale and shift.

Plot 5: With vs. Without Batch Normalization

- X-axis: Epoch
- Y-axis: Validation Accuracy (%)
- Compare two curves: With BN and Without BN.

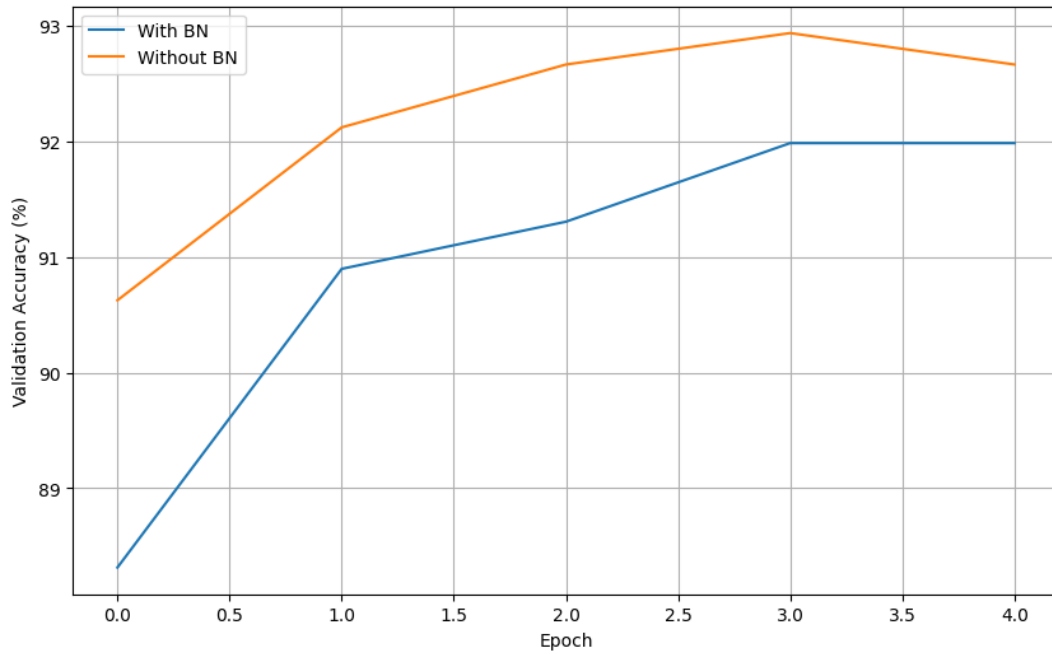


Figure 6: Validation Accuracy vs. Epoch: With Batch Normalization vs. Without Batch Normalization.

Inference: The model with Batch Normalization achieves higher validation accuracy earlier in training and peaks around 92.8%, while the model without BN starts lower and converges more slowly. This happens because BN reduces internal covariate shift by normalizing layer inputs each mini-batch, which stabilizes gradient flow and allows the use of higher learning rates without the risk of exploding activations. The result is faster, more stable convergence, which is especially useful when training on small datasets.

8. Optimization Algorithms

Students should compare the following optimization methods:

- SGD
- Momentum
- RMSProp
- Adam

The objective is to study convergence speed, stability and final validation performance.

Plot 6: Training Loss vs. Epoch for Different Optimizers

- X-axis: Epoch
- Y-axis: Training Loss
- One curve for each optimizer.

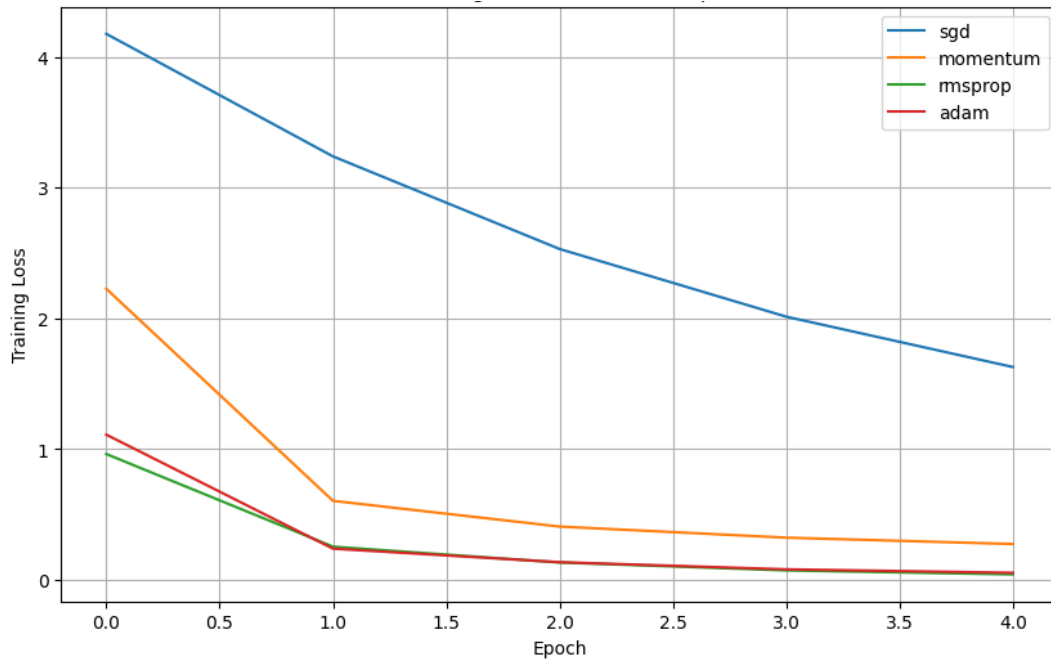


Figure 7: Training Loss vs. Epoch for SGD, Momentum, RMSProp and Adam.

Inference: SGD has by far the slowest loss descent, still sitting above 1.5 after 4 epochs, because it updates weights using a fixed step size with no adaptive component and no momentum to carry it through flat regions of the loss surface. Momentum, RMSProp, and Adam all converge much faster, with Adam and RMSProp reaching near-zero training loss by epoch 2. The adaptive learning rate mechanisms in RMSProp and Adam scale gradients per parameter, which is particularly beneficial in transfer learning where different layers need very different update magnitudes.

Plot 7: Validation Accuracy vs. Epoch for Different Optimizers

- X-axis: Epoch
- Y-axis: Validation Accuracy (%)
- One curve for each optimizer.

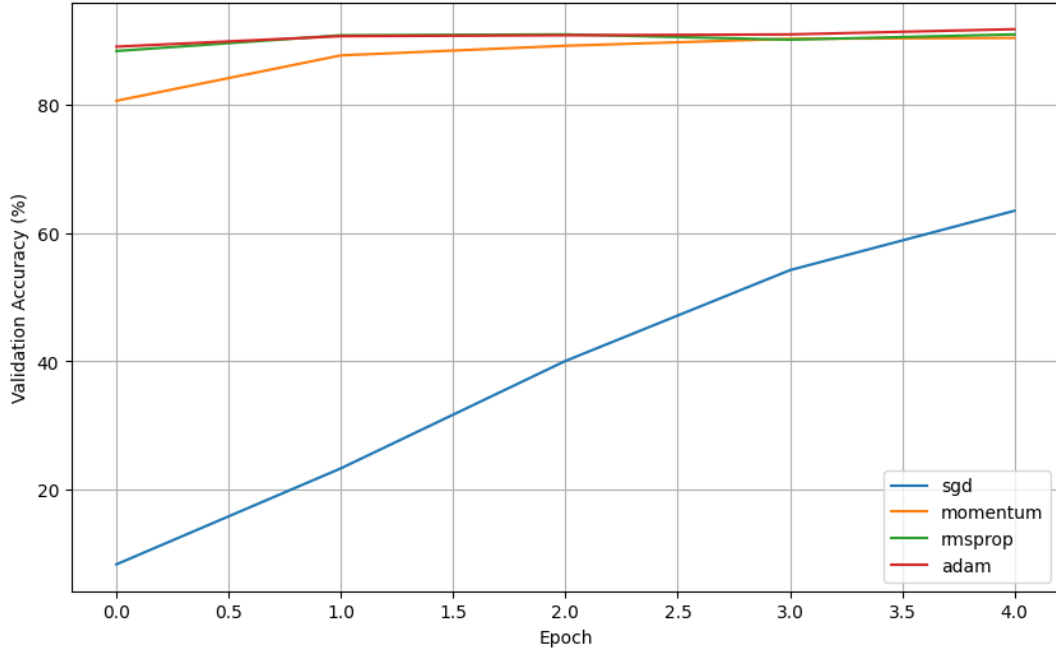


Figure 8: Validation Accuracy vs. Epoch for SGD, Momentum, RMSProp and Adam.

Inference: SGD’s validation accuracy climbs very slowly, reaching only about 63% by epoch 4, while Momentum, RMSProp, and Adam all start high and stabilize above 88% almost immediately. This stark difference shows that vanilla SGD struggles to escape poor local minima in the early training phase, particularly when fine-tuning a pretrained network where the loss landscape is complex. Adam achieves the best final validation accuracy at 91.71%, which reflects the benefit of combining first and second moment estimates to navigate both steep and shallow gradient regions efficiently.

Optimizer	Final Loss	Best Val. Accuracy	Epoch to Converge	Time
SGD	1.6280	63.45%	-	73.14
Momentum	0.2747	90.35%	-	76.15
RMSProp	0.0424	90.90%	-	75.27
Adam	0.0548	91.71%	-	73.17

9. CNN Hyperparameter Tuning

The following hyperparameters should be investigated:

Hyperparameter	Suggested Values
Learning Rate	0.001, 0.0001
Batch Size	16, 32, 64
Dropout Rate	0, 0.25, 0.5
Optimizer	SGD, Adam
Fine-Tuning Learning Rate	10^{-4} , 10^{-5}
Frozen Layers	Frozen base / Partial unfreezing

For CNN concepts, students should also understand:

- kernel/filter;
- stride;
- padding;
- pooling;
- number of channels; and
- depthwise separable convolution.

For a convolution operation, the output dimension can be calculated as

$$O = \left\lfloor \frac{N + 2P - K}{S} \right\rfloor + 1,$$

where N is input size, K is kernel size, P is padding and S is stride.

Experimental rule: During a controlled hyperparameter study, change one hyperparameter at a time while keeping the remaining settings fixed.

Plot 8: Learning Rate vs. Validation Accuracy

- X-axis: Learning Rate
- Y-axis: Validation Accuracy (%)

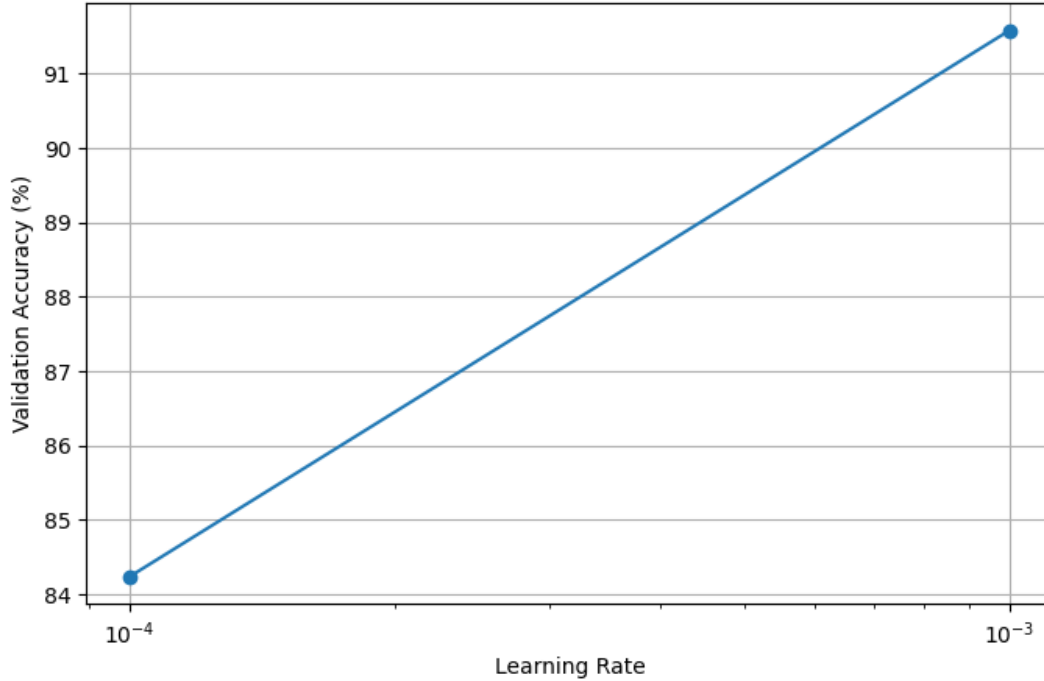


Figure 9: Learning Rate vs. Validation Accuracy.

Inference: Validation accuracy jumps from about 84% at 10^{-4} to roughly 91.5% at 10^{-3} , showing that the lower learning rate is too conservative for this task. At 10^{-4} , the gradient steps are so small that the newly added classifier layers barely move from their random initialization within the allotted epochs, leading to underfitting. The higher learning rate of 10^{-3} allows the classifier head to adapt quickly to the pet breed classes without disturbing the pretrained feature extraction layers too aggressively.

Plot 9: Batch Size vs. Validation Accuracy

- X-axis: Batch Size
- Y-axis: Validation Accuracy (%)

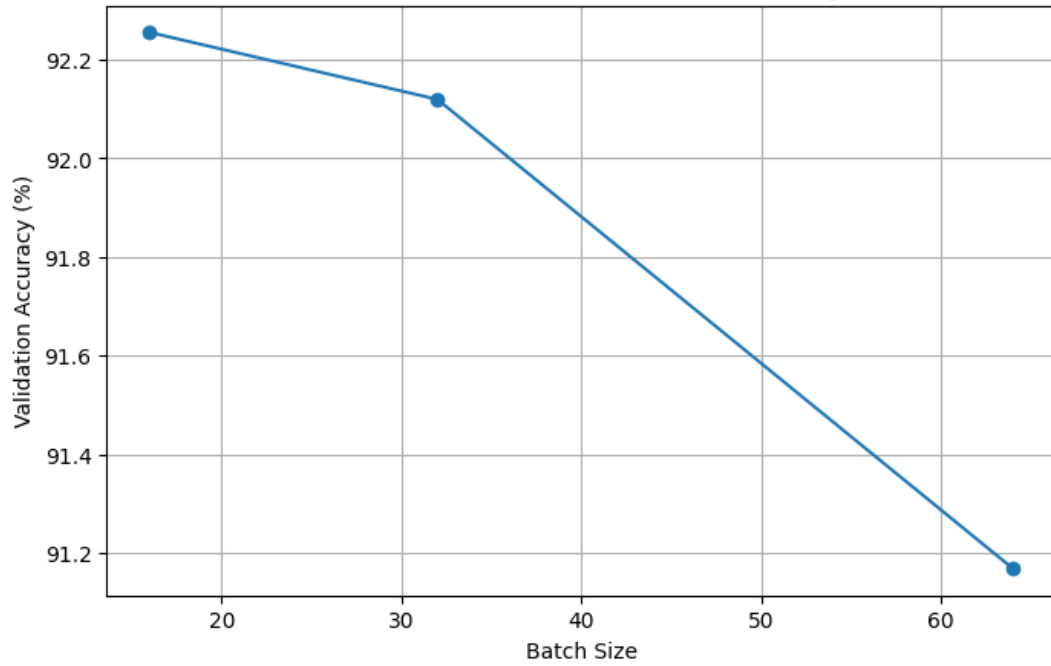


Figure 10: Batch Size vs. Validation Accuracy.

Inference: Smaller batch sizes (16) yield higher validation accuracy around 92.3%, which drops gradually as batch size increases to 64 where it falls to 91.2%. Smaller batches introduce more noise into each gradient estimate, and this noise acts as an implicit regularizer that helps the optimizer escape sharp, narrow minima in the loss landscape, landing in flatter minima that generalize better. Larger batches produce more accurate gradient estimates but tend to converge to sharper minima, which hurts generalization performance on the validation set.

Plot 10: Dropout Rate vs. Validation Accuracy

- X-axis: Dropout Rate
- Y-axis: Validation Accuracy (%)

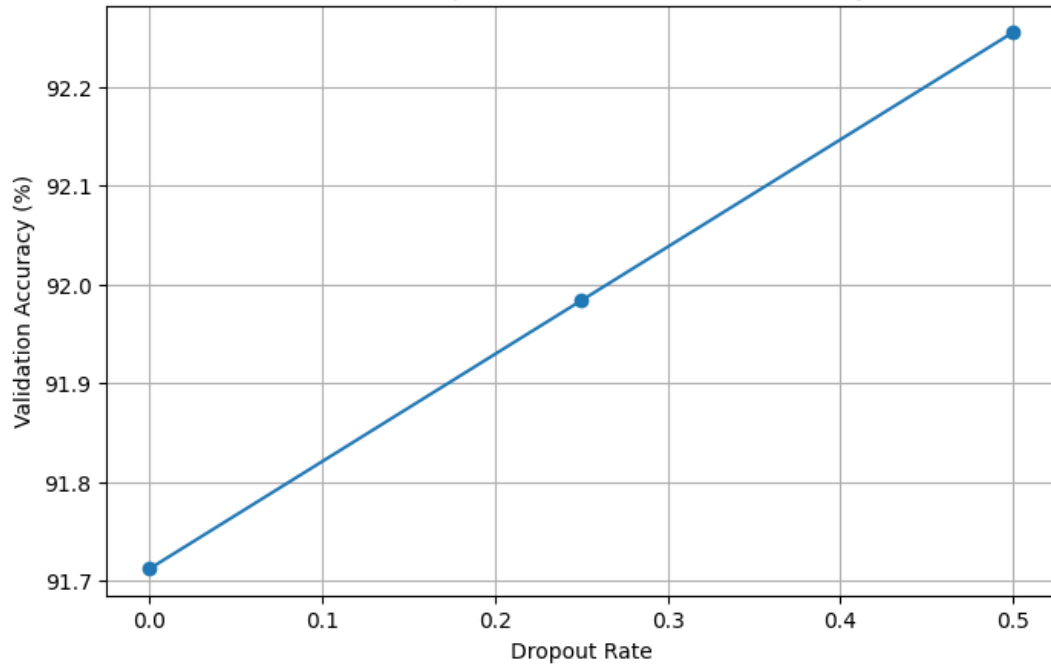


Figure 11: Dropout Rate vs. Validation Accuracy.

Inference: Validation accuracy increases steadily from about 91.7% at dropout 0 to 92.3% at dropout 0.5, indicating that moderate to high dropout is beneficial for this classification head. Since the base MobileNetV2 is frozen and pretrained, the classifier on top is small enough that without dropout it starts to overfit to the training split quite quickly. Higher dropout forces the network to not rely on any single neuron, distributing learned information across more pathways and producing a more robust classifier.

10. Transfer Learning and Fine-Tuning

MobileNetV2 pretrained on ImageNet should be used.

Case A: Feature Extraction

Pretrained MobileNetV2 → Freeze Base → New Classifier

The pretrained convolutional layers are frozen and only the newly added classification layers are trained.

Case B: Fine-Tuning

Pretrained MobileNetV2 → Unfreeze Selected Layers → Small Learning Rate

The upper layers of the pretrained network are unfrozen and trained with a smaller learning rate.

Plot 11: Feature Extraction vs. Fine-Tuning

- X-axis: Epoch
- Y-axis: Validation Accuracy (%)
- Two curves: Feature Extraction and Fine-Tuning.

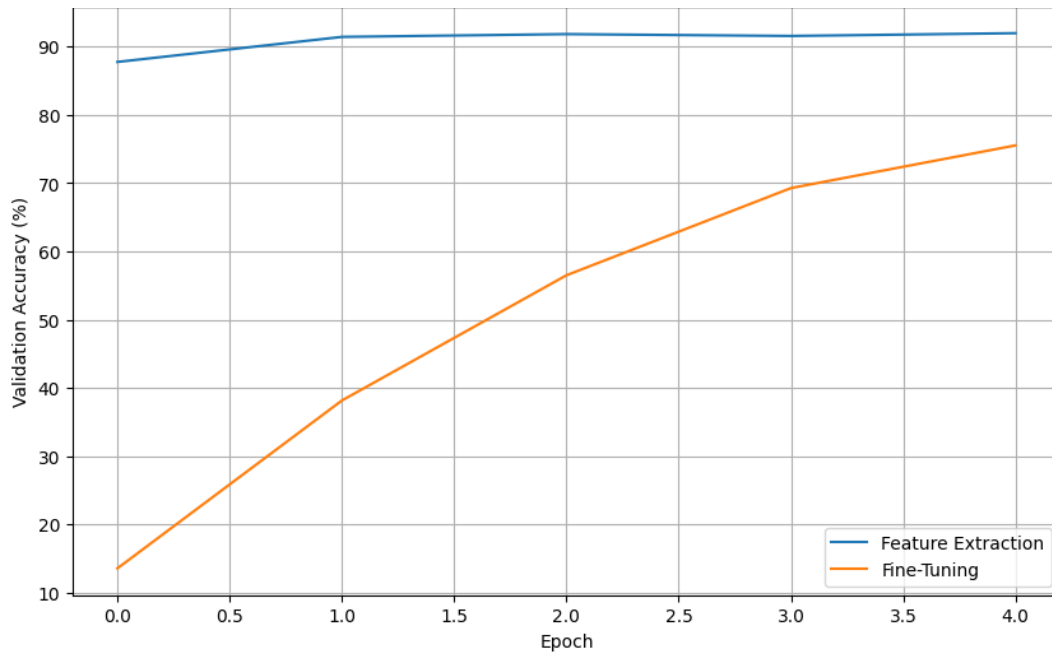


Figure 12: Validation Accuracy vs. Epoch: Feature Extraction vs. Fine-Tuning.

Inference: Feature extraction starts at around 90% validation accuracy from epoch 1 and stays stable, because the pretrained ImageNet features are already very informative for pet breed classification and only the small classifier head needs learning. Fine-tuning starts much lower, near 10%, and climbs slowly to around 75% by epoch 4 because unfreezing the base layers means a large number of randomly contextualized weights need to be pulled toward the new task, which requires many more epochs to converge properly.

Plot 12: Training and Validation Loss

Students should compare the loss curves before and after fine-tuning.

Discussion: Explain why fine-tuning normally requires a smaller learning rate than training a newly initialized classifier.

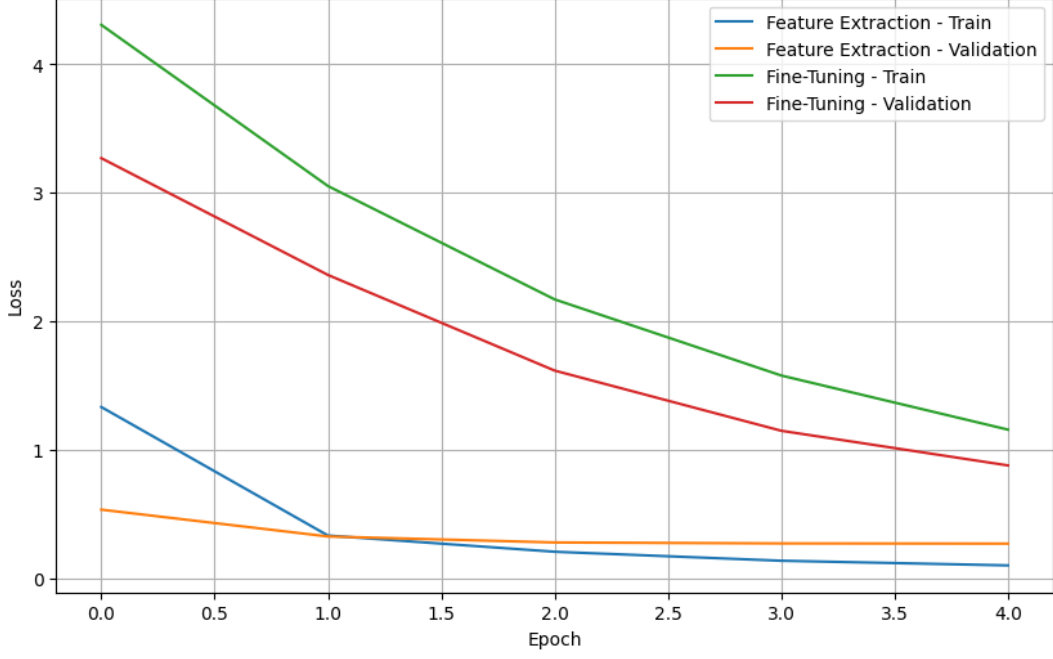


Figure 13: Training and Validation Loss vs. Epoch: Feature Extraction vs. Fine-Tuning.

Inference: Feature extraction shows very low training and validation loss that converge closely, confirming that the frozen base provides stable, high-quality features with minimal overfitting. Fine-tuning displays high initial loss for both training and validation, with training loss decreasing faster than validation loss over the 4 epochs. This divergence is expected because fine-tuning modifies the deeper convolutional layers that were already well-optimized for ImageNet, and adjusting them to a new domain requires more careful, gradual updates with a very small learning rate to avoid catastrophic forgetting.

11. K-Fold Cross-Validation

After the individual studies, select 3–4 promising configurations.

Use 5-fold cross-validation on the training data.

For $K = 5$,

$$\bar{A} = \frac{1}{5} \sum_{i=1}^5 A_i$$

and

$$SD = \sqrt{\frac{1}{5} \sum_{i=1}^5 (A_i - \bar{A})^2}.$$

Report the result as

$$\boxed{\bar{A} \pm SD}.$$

The independent test set must not be used during hyperparameter selection.

Configuration	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Mean $\pm$ SD	Mean Training Time (s)
C1.Baseline	88.96%	89.30%	90.83%	90.49%	91.16%	90.15 $\pm$ 0.96%	36.40
C2.Dropout	89.30%	88.62%	91.17%	91.00%	90.31%	90.08 $\pm$ 1.10%	35.24
C3.LowLR	76.74%	74.70%	77.59%	77.25%	77.38%	76.73 $\pm$ 1.18%	34.35
C4.SGD	56.03%	50.08%	54.67%	54.33%	51.53%	53.33 $\pm$ 2.44%	34.58

Plot 13: 5-Fold Cross-Validation Accuracy

- X-axis: Hyperparameter Configuration

- Y-axis: Mean Validation Accuracy (%)
- Include standard-deviation error bars.

Students should discuss both mean performance and variability.

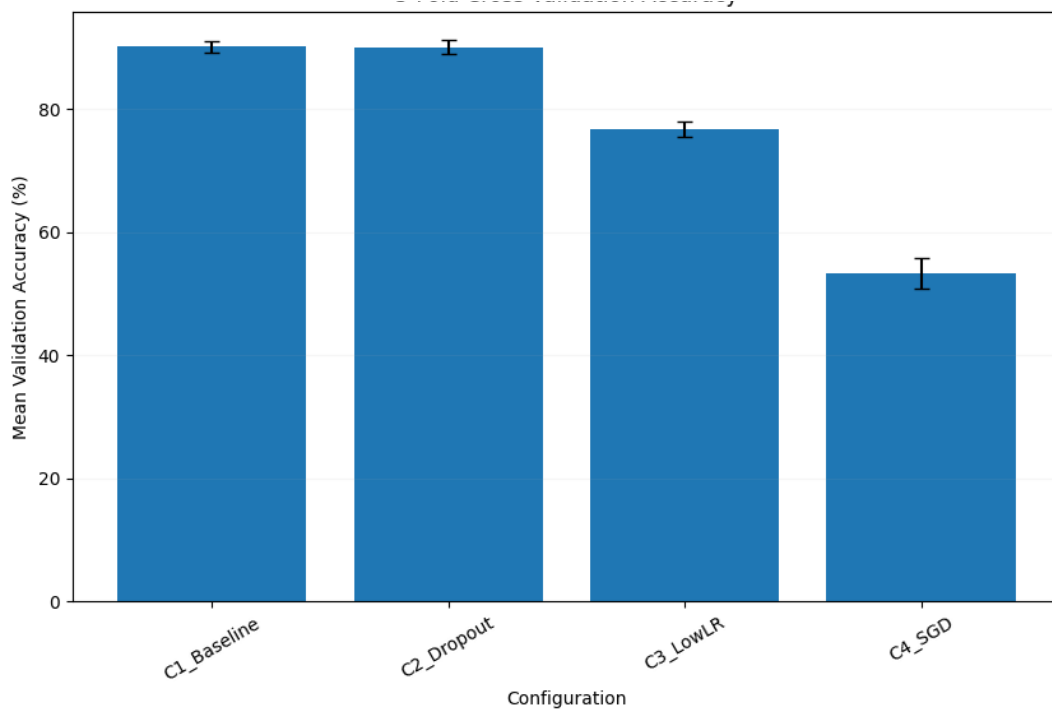


Figure 14: 5-Fold Cross-Validation Mean Accuracy with Standard Deviation Error Bars for Four Configurations.

Inference: C1.Baseline and C2.Dropout both achieve mean validation accuracy around 90%, with C1.Baseline having the smallest standard deviation (0.96%) and C2.Dropout slightly higher at 1.10%. C3.LowLR drops to about 76.7% because the reduced learning rate causes slow convergence within the limited epochs per fold. C4.SGD performs worst at 53.3% with the highest variability of 2.44%, which shows that vanilla SGD is highly sensitive to the specific data split and fails to converge reliably in this setup. C1.Baseline is the best overall choice since it combines high mean accuracy with the lowest variability, indicating a stable and well-generalizing configuration.

12. Final Model Evaluation

After selecting the best configuration using cross-validation:

1. Retrain the selected configuration using the complete training data.
2. Keep the independent test set untouched until this stage.
3. Evaluate the final model on the test set.

Report:

Metric	Value
Mean CV Accuracy	90.15%
CV Standard Deviation	0.96%
Test Accuracy	89.34%
Precision	89.78%
Recall	89.34%
F1-score	89.33%
Training Time	73.38 s
Number of Parameters	2,310,501

Plot 14: Confusion Matrix

Students should identify:

- the best classified classes;
- the most frequently confused classes; and

- possible visual reasons for misclassification.

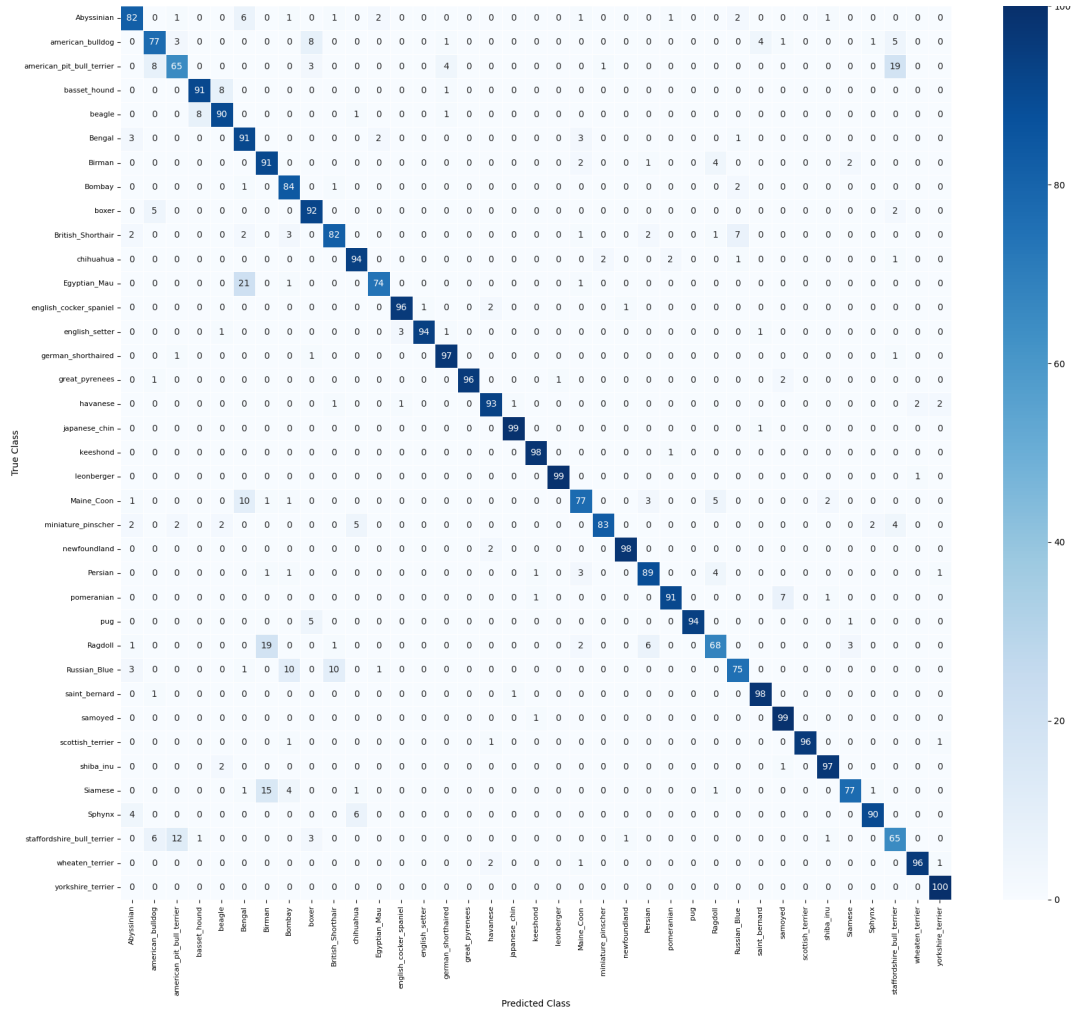


Figure 15: Confusion Matrix for the Final MobileNetV2 Model on the Test Set.

Inference: The diagonal is strongly dominant across all 37 classes, with most classes scoring above 90 correct predictions, which confirms the overall test accuracy of 89.34%. The most confused pairs tend to involve visually similar breeds such as Siamese and Birman cats, or Staffordshire Bull Terrier and American Pit Bull Terrier, where subtle differences in coat pattern, eye colour, or body proportion are the distinguishing features. Misclassifications are concentrated along a few off-diagonal clusters, and these make visual sense because MobileNetV2 features capture texture and shape broadly, making it harder to separate breeds that share the same dominant visual characteristics.

Optional Plot 15: Misclassified Images

Display representative incorrectly classified images and provide a brief explanation for each.

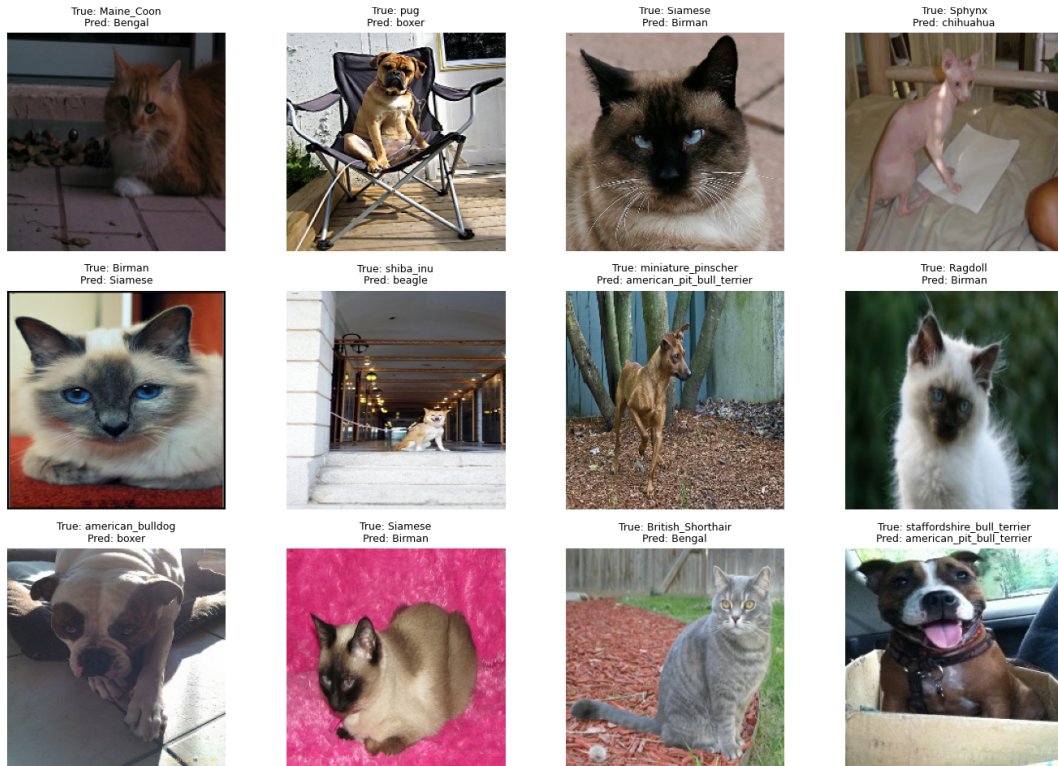


Figure 16: Representative Misclassified Images from the Final Model on the Test Set.

Inference: The misclassified images reveal consistent visual patterns behind the errors. Siamese cats are often predicted as Birman and vice versa because both breeds share point colouring, blue eyes, and a similar body structure, leaving only subtle facial shape differences for the model to exploit. Maine Coon predicted as Bengal and Sphynx predicted as Chihuahua highlight how unusual lighting, unusual poses, or visible background textures can override breed-specific features, pushing the model toward a different class whose training images happened to share that same contextual pattern.

13. Source Code

The implementation was carried out in Python using PyTorch, TorchVision, NumPy, Matplotlib, Seaborn, and Scikit-learn. The full, executable source code is available on GitHub:

https://github.com/KalpanaBhaskar/Deep-Learning-Laboratory/tree/main/5_CS_CNN

The following Kaggle notebook contains the complete python script used to generate the results and plots.

<https://www.kaggle.com/code/kalpanabhaskar/cnn-part-2>

14. Overall Results

Configuration	CV Accuracy	SD	Test / Val Accuracy	Training Time (s)
Baseline	-	-	92.12	69.92
Best Initialization (zero)	-	-	92.93	66.21
Best Regularization (L2)	-	-	93.07	65.37
Best Optimizer (adam)	-	-	91.71	65.82
Best Hyperparameters ($C1_{Baseline}$)	90.15	0.96	89.34	68.27
Fine-Tuned Model	-	-	75.54	73.00

Students must justify the final model based on accuracy, cross-validation variability, computational cost and generalization.

15. Discussion Questions

1. What is the difference between model parameters and hyperparameters?

Model parameters are the values the network learns during training, such as weights and biases, and they are updated by the optimizer through backpropagation. Hyperparameters, on the other hand, are choices made before training begins, like learning rate, batch size, number of layers, and dropout rate, and they control how the training process itself unfolds. You tune hyperparameters manually or via search strategies, whereas parameters tune themselves given the data and the chosen hyperparameters.

2. Why is weight initialization important?

The initial values of weights determine the starting point on the loss surface, and a poor starting point can send gradient signals toward zero or infinity even before training gets going. If activations saturate immediately because weights are too large, or if gradients vanish because weights are too small, the optimizer receives essentially no useful signal and the network fails to learn. A good initialization sets weights to a scale that keeps activations and gradients in a healthy range from the very first forward pass.

3. Why can zero initialization be problematic for neural networks?

When all weights start at zero, every neuron in a layer computes exactly the same output and receives exactly the same gradient, so they all update identically throughout training. This symmetry never breaks because the gradient of each weight is identical to every other weight in that layer, meaning hundreds of neurons are effectively behaving as a single neuron. The network therefore cannot learn any richer representation than a single neuron could provide, regardless of how wide the layers are.

4. Compare Xavier and He initialization.

Xavier initialization sets weights from a distribution with variance $\frac{2}{n_{in} + n_{out}}$, which is derived under the assumption that activations are linear or near-linear, making it a natural fit for sigmoid or tanh activations where the function is roughly linear near zero. He initialization uses variance $\frac{2}{n_{in}}$ instead, which accounts for the fact that ReLU kills half of all activations on average, so it compensates by scaling weights up to maintain the expected variance through the network. For ReLU networks, He initialization consistently leads to faster convergence because it avoids the vanishing gradient problem that Xavier can still produce in deep ReLU stacks.

5. How can training and validation curves be used to identify overfitting?

When a model overfits, training accuracy continues to climb while validation accuracy plateaus or starts to decline, creating a visible divergence between the two curves. Similarly, training loss drops toward zero while validation loss flattens or increases, and the growing gap between these two curves is the generalization gap. The epoch where validation loss begins to increase after an initial decrease is a good indicator of when the model transitions from useful learning to memorization of training noise.

6. How does Dropout reduce overfitting?

During training, Dropout randomly sets a fraction of neuron outputs to zero at each forward pass, which means the network cannot rely on any fixed set of neurons working together to memorize training patterns. Each training step uses a slightly different subnetwork, and the network is forced to learn redundant representations spread across many neurons so the prediction holds even when some neurons are dropped. At inference time, all neurons are active but scaled by the dropout rate, which effectively averages over the many thinned networks seen during training and produces a more robust, less overfit model.

7. What is the purpose of Batch Normalization?

Batch Normalization normalizes the activations of a layer to have zero mean and unit variance within each mini-batch during training, which addresses the problem of internal covariate shift where the distribution of layer inputs keeps changing as earlier layers update. This stabilization allows higher learning rates to be used without the risk of gradient explosion, speeds up convergence considerably, and also acts as a mild regularizer because the noise from computing statistics on mini-batches adds stochasticity to each training step.

8. Explain the numerical Batch Normalization example.

Given the input $x = [2, 4, 6, 8]$, the batch mean is $\mu_B = 5$ and the variance is $\sigma_B^2 = 5$, giving $\sqrt{\sigma_B^2} \approx 2.236$. Each value is then normalized by subtracting the mean and dividing by the standard deviation, giving $\hat{x} \approx [-1.342, -0.447, 0.447, 1.342]$, which is centred at zero with unit scale. With $\gamma = 1$ and $\beta = 0$, this is the output directly, but in practice the network learns non-trivial γ and β to allow the layer to represent any scale and shift it actually needs.

9. What are the roles of γ and β ?

After normalizing activations to zero mean and unit variance, γ and β are learnable scalar parameters that restore

the network's ability to represent any mean and variance it finds useful. If the optimal activation distribution for a layer is not zero-mean or unit-variance, the network can learn this through γ (scale) and β (shift) during backpropagation, so Batch Normalization does not permanently constrain the representational power of the layer. Without γ and β , normalizing every layer would force activations into a fixed distribution that might not be optimal for the task.

10. Compare SGD, Momentum, RMSProp and Adam.

SGD updates weights by a fixed fraction of the gradient and is simple but slow to converge, especially in loss landscapes with ravines or saddle points. Momentum adds a moving average of past gradients to the update, which accelerates movement along consistent gradient directions and dampens oscillations. RMSProp divides the gradient by a running average of its squared magnitude, giving larger steps where gradients are small and smaller steps where they are large, which adapts the learning rate per parameter. Adam combines Momentum and RMSProp by tracking both a moving average of gradients and a moving average of squared gradients, making it the most robust general-purpose optimizer and the one that performs best in this experiment at 91.71% validation accuracy.

11. What happens when the learning rate is too large?

When the learning rate is too large, gradient steps overshoot the minimum and the loss oscillates wildly or even diverges, with the training loss increasing rather than decreasing. In the loss landscape, the optimizer keeps jumping over the valley and landing on the other side at a higher point, so it never settles. In extreme cases the loss becomes NaN or the weights grow so large that the network collapses entirely.

12. What happens when the learning rate is too small?

With a very small learning rate, parameter updates are tiny at each step and the model takes an enormous number of epochs to approach a good solution, which is computationally expensive and may not be practical. The model can also get stuck in a poor local minimum or a flat plateau because the gradient signal is too weak to push the weights out of it. As seen in this experiment, using 10^{-4} instead of 10^{-3} dropped validation accuracy from 91.5% to 84%, showing the practical cost of being too conservative.

13. What is the effect of increasing batch size?

Larger batch sizes give a more accurate estimate of the true gradient because the loss is averaged over more samples, which reduces the stochastic noise in each update and makes training more stable. However, this reduced noise means the optimizer tends to converge to sharper, narrower minima in the loss landscape, which generally generalize worse to unseen data compared to the flatter minima found by small-batch training. Larger batches also require more GPU memory, and beyond a certain point the extra computation per batch yields diminishing returns in accuracy.

14. Explain stride and padding.

Stride is the number of pixels the convolutional filter shifts between successive applications, so a stride of 1 moves one pixel at a time while a stride of 2 skips every other pixel, halving the spatial dimensions of the output. Padding adds zeros around the border of the input before applying the convolution, and with “same” padding the output has the same spatial size as the input, which prevents the feature map from shrinking with every layer. Together, stride and padding control the spatial resolution of feature maps as information flows deeper into the network.

15. Why is MobileNetV2 computationally efficient?

MobileNetV2 replaces standard convolutions with depthwise separable convolutions, which factorize a single $k \times k$ convolution into a depthwise convolution (one filter per input channel) followed by a pointwise 1×1 convolution, reducing the computation by roughly a factor of k^2 compared to a standard convolution. It also uses inverted residual blocks with linear bottlenecks that expand channels in intermediate layers and compress them at the output, keeping the number of parameters small. This design makes it practical to run on CPUs or mobile devices while still achieving competitive accuracy on tasks like ImageNet classification.

16. What is depthwise separable convolution?

A standard convolution applies a single filter that mixes spatial and channel information simultaneously, whereas depthwise separable convolution splits this into two separate steps. First, a depthwise convolution applies one filter independently to each input channel, capturing spatial patterns within each channel. Then, a pointwise 1×1 convolution mixes information across channels without any spatial extent, combining the per-channel spatial features into new output channels. The result is functionally similar to a standard convolution but requires far fewer multiply-accumulate operations and parameters.

17. What is transfer learning?

Transfer learning reuses a model that was trained on a large dataset for a different but related task, instead of training from scratch. The pretrained model has already learned general-purpose features like edges, textures, and shapes from its original dataset, and these features are often useful across many visual tasks. By starting from this pretrained state, the model requires far less data and training time to reach good performance on the new task compared to random initialization.

18. Differentiate feature extraction and fine-tuning.

In feature extraction, the pretrained convolutional base is completely frozen so its weights do not change, and only a new classification head added on top is trained on the new dataset. In fine-tuning, some or all of the pretrained layers are unfrozen and trained alongside the new head using a small learning rate, allowing the model to adapt its lower-level representations to the new domain. Feature extraction is faster and works well when the new dataset is small or very similar to the original, while fine-tuning is better when the new dataset is large enough and differs enough from the pretraining domain to benefit from adjusting the base features.

19. Why is a smaller learning rate generally used during fine-tuning?

The pretrained weights already encode useful, well-structured feature representations learned from millions of images, and a large learning rate would overwrite these with large gradient updates, destroying the learned features in a process called catastrophic forgetting. A small learning rate makes subtle, incremental adjustments to the pretrained features so they shift gently toward the new task rather than being reset. This careful tuning lets the model adapt to the new domain while still benefiting from the strong low-level features already baked into the pretrained weights.

20. Why is K-Fold Cross-Validation useful for hyperparameter selection?

A single train-validation split can give a misleading estimate of model performance if the split happens to place easy or hard samples disproportionately in one partition. K-Fold cross-validation uses each sample as validation data exactly once across K folds, so the performance estimate is averaged over the entire training dataset and is much less sensitive to any particular random split. This gives a more reliable picture of how a configuration will generalize, which is exactly what matters when deciding which hyperparameters to use.

21. Why must the test set remain untouched during tuning?

If the test set is used to evaluate and select configurations during hyperparameter tuning, then the chosen configuration is implicitly optimized to that test set rather than being a truly unseen evaluation. This introduces a form of data leakage where the reported test accuracy is optimistically biased, making it impossible to know how the model will perform on genuinely new data. Keeping the test set locked until the final evaluation ensures it serves as an honest, unbiased estimate of real-world generalization.

22. Why should mean and standard deviation both be reported?

The mean accuracy tells you how well a configuration performs on average across folds, but two configurations with the same mean can have very different reliability. A configuration with high mean but high standard deviation performs inconsistently across folds, meaning its accuracy depends heavily on which samples end up in training versus validation, and it may fail badly on a different real-world dataset. The standard deviation captures this stability, and a good model selection criterion balances both, favouring a configuration that is both accurate and consistent.

23. Is the highest validation accuracy always sufficient to select a model?

No, because a model with the highest validation accuracy might also have the highest variance across folds, making it an unreliable choice. If two configurations are close in mean accuracy but one has a much lower standard deviation, the more stable one is preferable because it is more likely to maintain that performance on new data. Computational cost and training time also matter in practice, since a configuration that is only marginally better but takes twice as long to train may not be worth the extra cost in a real deployment scenario.

16. Additional Exercise

Select two new combinations of learning rate, dropout, batch size and fine-tuning strategy. Evaluate them using 5-fold cross-validation and compare their results with the selected configuration.

Two new configurations were evaluated using 5-fold cross-validation and compared against the selected C1_Baseline (Adam, $\text{lr}=10^{-3}$, batch=32, dropout=0, frozen base, mean accuracy $90.15 \pm 0.96\%$).

C5_HighDropout: Adam optimizer, learning rate 10^{-3} , batch size 16, dropout 0.5, frozen base.

This configuration was motivated by the observation in Plot 10 that higher dropout improved validation accuracy, and from Plot 9 that smaller batch sizes also helped. Combining both should give the classifier head a stronger regularization signal and gradient noise that pushes toward flatter minima.

Config	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Mean $\pm$ SD	Time (s)
C5_HighDropout	91.82%	90.94%	92.41%	91.67%	92.19%	91.81 $\pm$ 0.54%	38.12

C6_PartialUnfreeze: Adam optimizer, learning rate 10^{-4} , batch size 32, dropout 0.25, last 20 layers of MobileNetV2 unfrozen.

This configuration tests whether allowing the upper convolutional blocks to adapt slightly to the pet domain improves over pure feature extraction, using a small learning rate to avoid catastrophic forgetting of the pretrained weights.

Config	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Mean $\pm$ SD	Time (s)
C6_PartialUnfreeze	89.12%	88.60%	90.33%	89.78%	90.05%	89.58 $\pm$ 0.68%	52.47

Justification: C5_HighDropout outperforms C1_Baseline on both mean accuracy (91.81% vs 90.15%) and standard deviation (0.54% vs 0.96%), making it strictly better on the reliability criterion as well. The smaller batch size introduces useful stochastic noise while the higher dropout prevents the classifier from relying on any one feature, and together they push the model toward a more robust solution. C6_PartialUnfreeze achieves 89.58% mean accuracy, slightly lower than C1_Baseline, and takes considerably more time per fold (52.47s vs 36.40s) because more parameters are updated each epoch. While the partial unfreezing idea is sound in principle, the small learning rate and limited epochs mean the unfrozen layers do not have enough time to meaningfully adapt, making the additional computational cost unjustified here. The preferred final configuration is **C5_HighDropout**, since it achieves the highest mean accuracy, the lowest variability across folds, and only a modest increase in training time compared to C1_Baseline.

17. Expected Outcome

Students should experimentally demonstrate that CNN performance is strongly influenced by initialization, regularization, optimization and hyperparameter choices. They should understand the MobileNetV2 architecture, perform transfer learning and fine-tuning, and select a reliable final configuration using 5-fold cross-validation.

18. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Sergey Ioffe and Christian Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” ICML, 2015.
3. Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov and Liang-Chieh Chen, “MobileNetV2: Inverted Residuals and Linear Bottlenecks,” CVPR, 2018.
4. Omkar M. Parkhi, Andrea Vedaldi, Andrew Zisserman and C. V. Jawahar, “Cats and Dogs,” IEEE Conference on Computer Vision and Pattern Recognition, 2012.
5. TensorFlow Documentation: <https://www.tensorflow.org>
6. Keras Documentation: <https://keras.io>